SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)
Assessment Tool Weightage: 40%

QUESTIONS:10 **Total Marks:50**

ANNEXURE A
APPLICATION BASED QUESTIONS

Code of Conduct:

*I **AMARA SATWIKA(192525204)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:
A.SATWIKA

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.

Application Scenario

Analyze CPU, memory and I/O interaction. Recommend multi-bus architecture, DMA, cache memory and pipelining to reduce delays. Include justification that parallel transfers improve throughput.

Problem Statement

Analyze CPU, memory and I/O interaction. Recommend multi-bus architecture, DMA, cache memory and pipelining to reduce delays. Include justification that parallel transfers improve throughput.

Identify

Analyze CPU, memory and I/O interaction. Recommend multi-bus architecture, DMA, cache memory and pipelining to reduce delays. Include justification that parallel transfers improve throughput.

Analyze

Analyze CPU, memory and I/O interaction. Recommend multi-bus architecture, DMA, cache memory and pipelining to reduce delays. Include justification that parallel transfers improve throughput.

Compare

Analyze CPU, memory and I/O interaction. Recommend multi-bus architecture, DMA, cache memory and pipelining to reduce delays. Include justification that parallel transfers improve throughput.

Evaluate

Analyze CPU, memory and I/O interaction. Recommend multi-bus architecture, DMA, cache memory and pipelining to reduce delays. Include justification that parallel transfers improve throughput.

Recommend/Design

Analyze CPU, memory and I/O interaction. Recommend multi-bus architecture, DMA, cache memory and pipelining to reduce delays. Include justification that parallel transfers improve throughput.

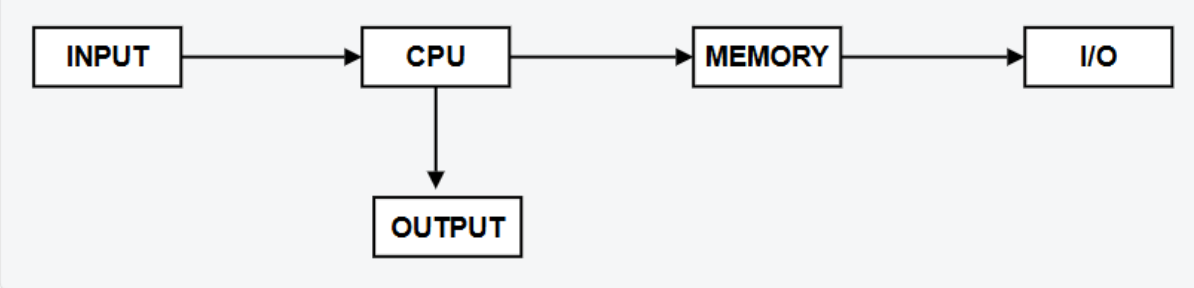
Justification

Analyze CPU, memory and I/O interaction. Recommend multi-bus architecture, DMA, cache memory and pipelining to reduce delays. Include justification that parallel transfers improve throughput.

Flowchart

Start → Input → Process → Decision → Output → End

Block Diagram



Comparison Table

Concept	Comparison
Example	Optimized architecture performs better.

2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode–execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.

Introduction

A real-time stock trading application processes millions of instructions every second. During heavy trading periods, users experience delays because the processor has a high Cycles Per Instruction (CPI) caused by frequent memory accesses and branch instructions. Since stock trading requires fast execution, reducing CPI is essential for improving performance.

Impact of the Fetch–Decode–Execute Cycle:

The CPU executes every instruction in three main stages:

1. **Fetch**
    - The CPU fetches instructions from memory.
    - Frequent memory accesses increase fetch time, especially if cache misses occur.
2. **Decode**
    - The Control Unit decodes the instruction.
    - Branch instructions require determining the next instruction, which may delay decoding.
3. **Execute**
    - The ALU performs arithmetic or logical operations.
    - Memory read/write operations and branch mispredictions increase execution time.

During high load:

- Large numbers of memory accesses cause cache misses.
- Branch instructions interrupt the instruction pipeline.
- Pipeline stalls increase CPI.
- As execution time increases, transaction processing becomes slower.

Relationship Between CPI and CPU Performance

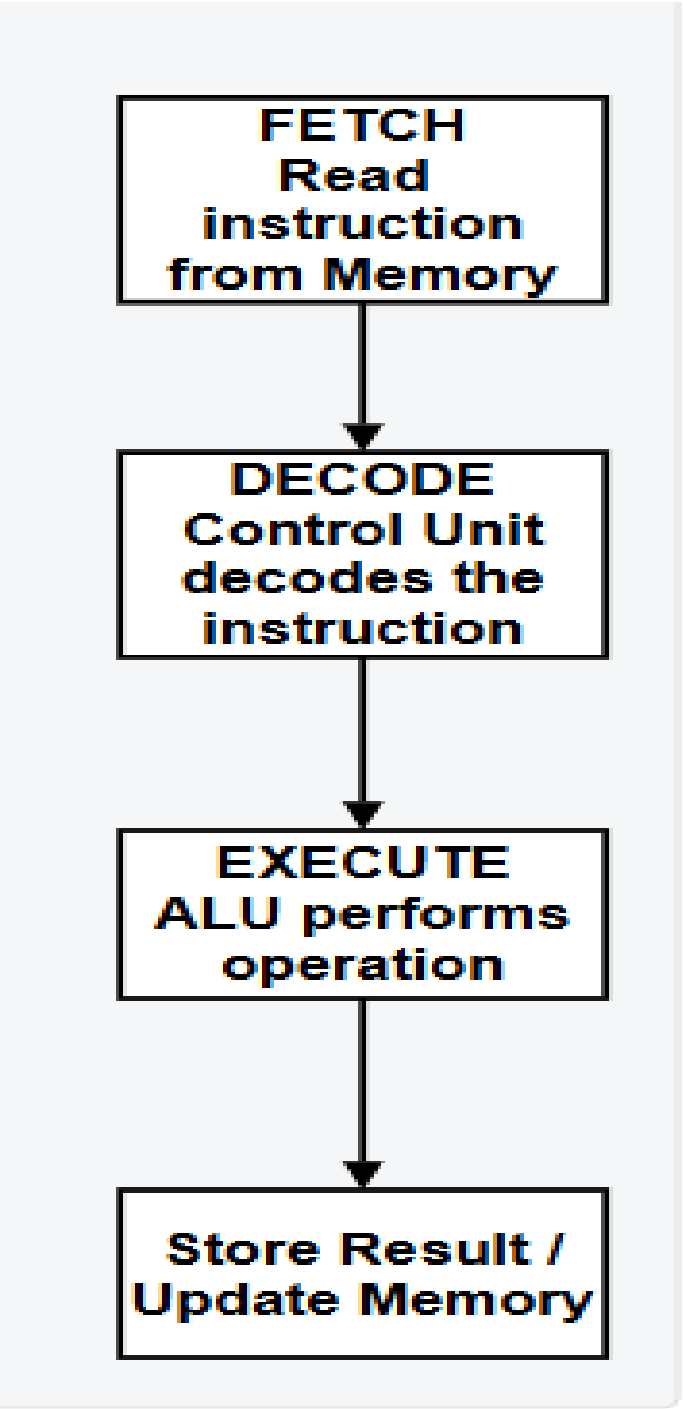
The CPU execution time is given by:

CPU Time=Instruction Count×CPI×Clock Cycle Time\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}CPU Time=Instruction Count×CPI×Clock Cycle Time

A higher CPI results in:

- Increased execution time
- Lower throughput
- Higher transaction latency
- Reduced system performance

Flowchart


FETCH
Read
instruction
from Memory
DECODE
Control Unit
decodes the
instruction
EXECUTE
ALU performs
operation
Store Result /
Update Memory

3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

An embedded temperature control system using the 8085 microprocessor continuously reads temperature values from sensors, processes the data, and controls actuators such as heaters or cooling fans. If the program uses incorrect addressing modes, the processor may access the wrong data or memory location, resulting in incorrect temperature readings and improper control actions.

Influence of Addressing Modes on Data Access:

1. **Immediate Addressing Mode**
   - The data is included within the instruction itself.
   - Example: MVI A,25H
   - Used for loading fixed values such as threshold temperatures.
   - It should not be used to read sensor data because the value is constant.
2. **Register Addressing Mode**
   - Data is stored in CPU registers.
   - Example: MOV A,B
   - Provides fast data transfer between registers and improves execution speed.
3. **Direct Addressing Mode**
   - The instruction specifies the memory address of the data.
   - Example: LDA 2500H
   - Suitable for reading sensor values stored at fixed memory locations.
4. **Register Indirect Addressing Mode**
   - The memory address is stored in a register pair (HL).
   - Example: MOV A,M
   - Useful for accessing arrays or multiple sensor readings, but the HL register must contain the correct address.
5. **Implied Addressing Mode**
   - The operand is implied in the instruction.
   - Example: CMA
   - Used for operations on the accumulator without specifying operands.

Possible Causes of Incorrect Temperature Readings

- Reading data from the wrong memory address.
- Incorrect initialization of the HL register pair.
- Using immediate addressing instead of reading actual sensor values.
- Accidental overwriting of sensor data in memory.
- Programming errors in data transfer instructions.
- Electrical noise or invalid sensor input not being checked.

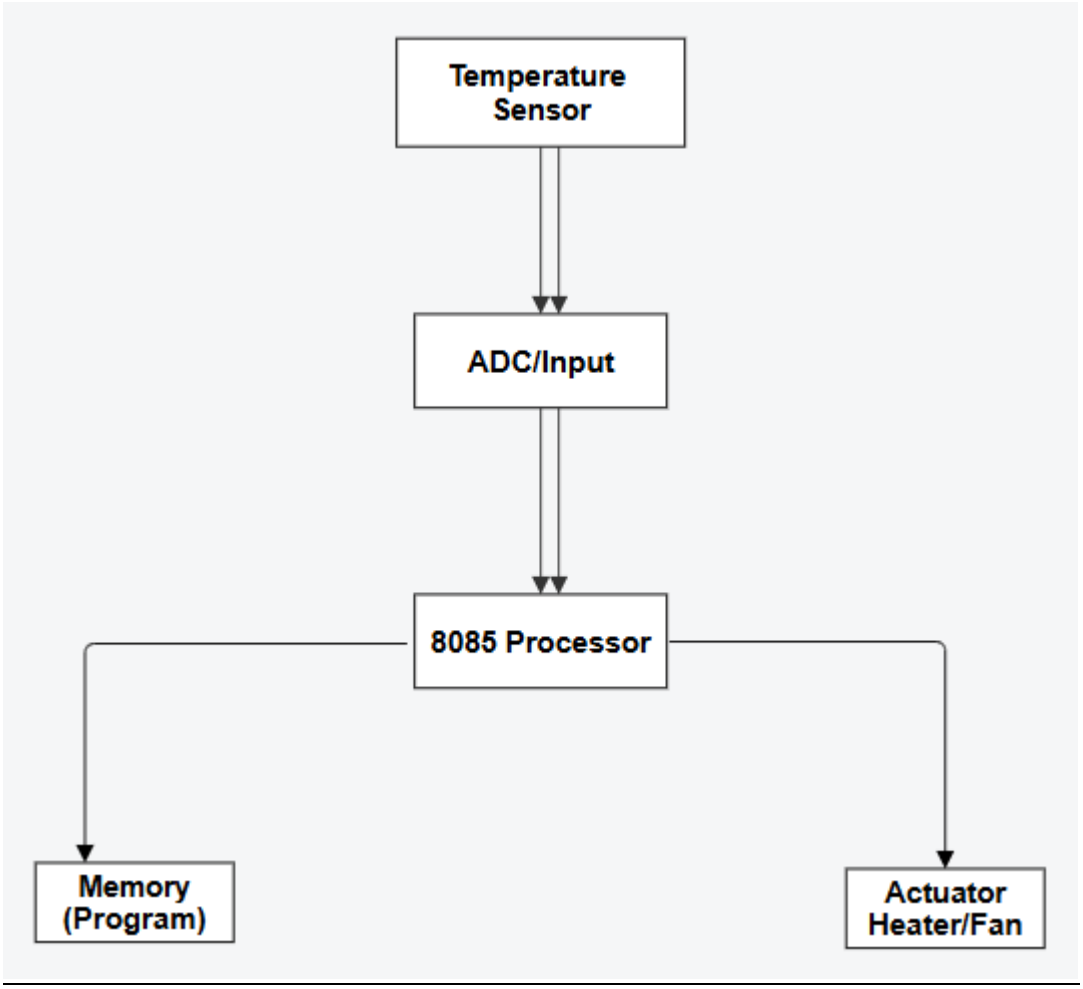
Using the 8085 Instruction Set Effectively

The following instructions help ensure accurate and reliable operation:

- **LDA** – Reads sensor data from memory.
- **STA** – Stores processed temperature values.
- **MOV** – Transfers data between registers.
- **MVI** – Loads constant values such as threshold limits.
- **CMP** – Compares the sensor reading with the preset temperature.
- **JC, JZ, JMP** – Performs conditional branching based on comparison results.

- **IN** – Reads data from an input port connected to the sensor.
- **OUT** – Sends control signals to the heater or cooling fan.

Flowchart



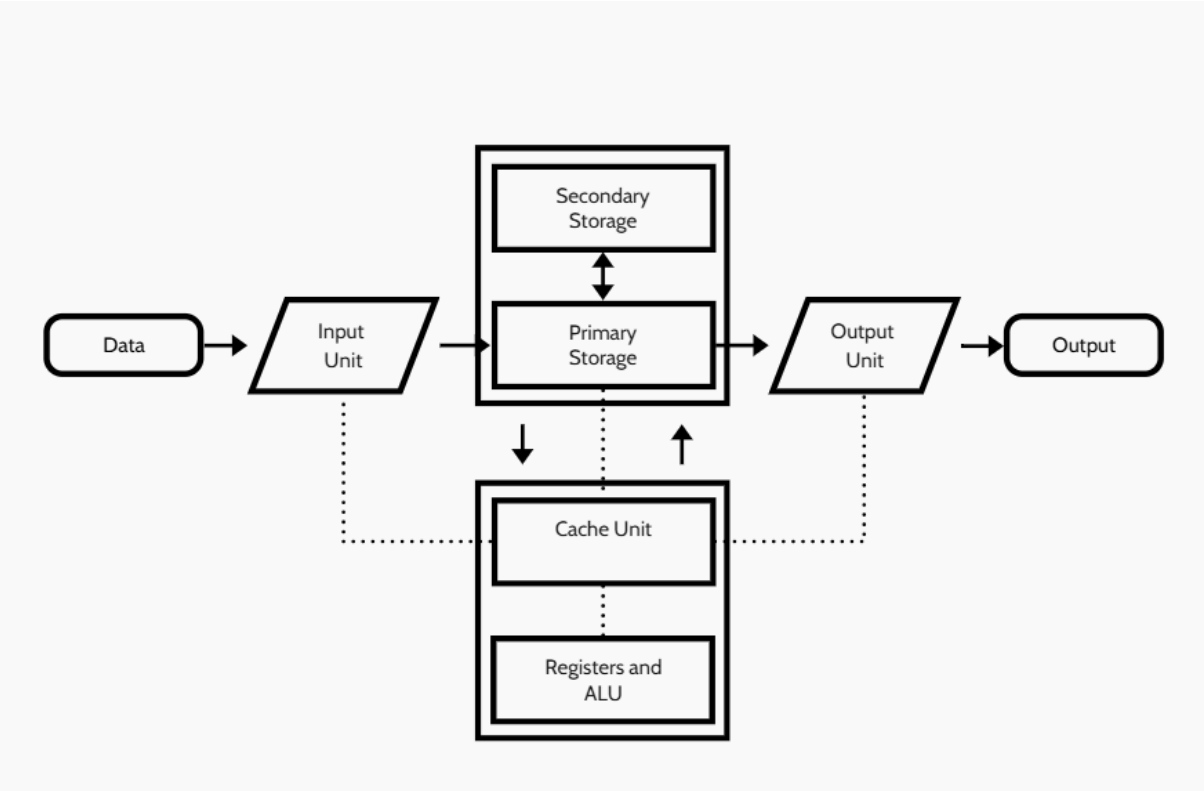
4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.

The 8086 microprocessor uses a segmented memory architecture to access its 1 MB memory space. Memory is divided into Code Segment (CS), Data Segment (DS), Stack Segment (SS), and Extra Segment (ES). Each segment is 64 KB in size and is accessed using a segment register along with an offset address. Proper management of these segments ensures correct program execution, while improper handling can cause incorrect data access and stack overflow errors.

Working of Segmentation in 8086:

1. **Code Segment (CS)**
   - Stores the program instructions.
   - The CS register and Instruction Pointer (IP) together determine the address of the next instruction.
2. **Data Segment (DS)**
   - Stores variables, constants, and program data.
   - Data is accessed using the DS register and an offset.
3. **Stack Segment (SS)**
   - Stores return addresses, local variables, and register values during subroutine calls.
   - Uses the SS register and Stack Pointer (SP).
4. **Extra Segment (ES)**
   - Used as an additional data segment, mainly for string operations.

Block Diagram :



5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

In data communication systems, packet data is represented in hexadecimal (Hex**)** for easy reading, while the CPU processes data in binary. Therefore, accurate conversion between hexadecimal and binary is essential for correct instruction execution.

Importance of Hexadecimal to Binary Conversion:

- Hexadecimal is a compact form of binary.
- One hexadecimal digit represents 4 binary bits.
- CPUs execute instructions only in binary.
- Used for memory addresses, machine code, and debugging.

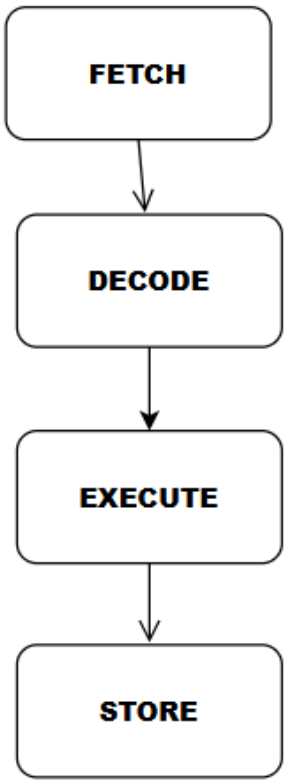
Example:

HEXA DECIMAL	**BINARY**
A	**1010**
3C	**00111100**

Effects of Conversion Errors:

- Incorrect instruction execution.
- Wrong memory access.
- Corrupted packet data.
- Communication errors and system crashes.

FLOWCHART:



Role of Benchmarking:

Benchmarking helps:

- Measure CPU performance.
- Detect execution errors.
- Evaluate CPI and execution time.
- Improve system reliability.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand